Métodos de Análisis de Datos 1

Unidad 5: Predicción y Comunicación de Resultados

Teoría Completa

Departamento de Matemática
Universidad Nacional del Sur

Primer Semestre 2026

Contents

1 Introducción **2**

2 Ciclo de Vida de Modelos en Producción **2**
2.1 Fases del Ciclo de Vida . . . 2
2.1.1 1. Desarrollo e Investigación . . . 2
2.1.2 2. Deployment (Puesta en Producción) . . . 2
2.1.3 3. Monitoreo . . . 3
2.1.4 4. Reentrenamiento y Versionado . . . 3
2.2 Data Drift y Concept Drift . . . 4
2.2.1 Data Drift . . . 4
2.2.2 Concept Drift . . . 4
2.3 Consideraciones de Deployment . . . 4
2.3.1 Escalabilidad . . . 4
2.3.2 Mantenibilidad . . . 5

3 Serialización de Modelos **5**
3.1 Formatos de Serialización . . . 5
3.1.1 pickle . . . 5
3.1.2 joblib . . . 6
3.1.3 ONNX (Open Neural Network Exchange) . . . 6
3.2 Buenas Prácticas de Serialización . . . 7

4 Interpretabilidad de Modelos **7**
4.1 Modelos Interpretables vs Caja Negra . . . 8
4.1.1 Modelos Inherentemente Interpretables . . . 8
4.1.2 Modelos de Caja Negra . . . 8
4.2 Trade-off Interpretabilidad vs Performance . . . 8
4.3 SHAP (SHapley Additive exPlanations) . . . 9
4.3.1 Idea Principal . . . 9
4.3.2 Interpretación de SHAP Values . . . 9
4.3.3 Tipos de Gráficos SHAP . . . 9
4.3.4 Uso en Python . . . 10
4.4 LIME (Local Interpretable Model-agnostic Explanations) . . . 10
4.4.1 Idea Principal . . . 10
4.4.2 Ventajas de LIME . . . 10

4.4.3 Desventajas de LIME . . . 11
4.4.4 Uso en Python . . . 11
4.5 Partial Dependence Plots (PDP) . . . 11
4.5.1 Cálculo . . . 11
4.5.2 Interpretación . . . 11
4.5.3 Limitaciones . . . 12
4.5.4 Uso en Python . . . 12
4.6 Feature Importance . . . 12
4.6.1 Importancia en Árboles y Random Forest . . . 12
4.6.2 Permutation Importance . . . 13

5 Comunicación de Resultados 13
5.1 Principios de Comunicación Efectiva . . . 14
5.1.1 1. Conocer a la Audiencia . . . 14
5.1.2 2. Estructura Clara . . . 14
5.1.3 3. Visualizaciones Efectivas . . . 15
5.1.4 4. Narrar una Historia . . . 15
5.2 Reportar Incertidumbre . . . 16
5.3 Lenguaje Claro y Preciso . . . 16

6 Reportes Reproducibles 16
6.1 Jupyter Notebooks . . . 16
6.1.1 Ventajas . . . 16
6.1.2 Buenas Prácticas . . . 17
6.1.3 Estructura Recomendada . . . 17
6.1.4 Exportación . . . 17
6.2 R Markdown . . . 17
6.2.1 Estructura . . . 18
6.2.2 Opciones de Chunks . . . 18
6.2.3 Formatos de Salida . . . 18
6.2.4 Parámetros . . . 19
6.3 Control de Versiones . . . 19
6.4 Gestión de Dependencias . . . 20

7 Ética en Análisis de Datos 20
7.1 Sesgo en Datos y Modelos . . . 21
7.1.1 Fuentes de Sesgo . . . 21
7.1.2 Consecuencias del Sesgo . . . 21
7.1.3 Mitigación de Sesgo . . . 21
7.2 Fairness y Discriminación Algorítmica . . . 22
7.2.1 1. Demographic Parity (Paridad Demográfica) . . . 22
7.2.2 2. Equalized Odds . . . 22
7.2.3 3. Predictive Parity . . . 22
7.2.4 Tensiones entre Definiciones . . . 22
7.3 Privacidad y Protección de Datos . . . 23
7.3.1 Principios . . . 23
7.3.2 Regulaciones . . . 23
7.3.3 Técnicas de Privacidad . . . 23
7.4 Responsabilidad en Uso de Modelos Predictivos . . . 24
7.4.1 Limitaciones y Comunicación . . . 24
7.4.2 Decisiones Humanas vs Algorítmicas . . . 24
7.4.3 Auditoría y Rendición de Cuentas . . . 25

7.5 Consideraciones Éticas en Proyectos 25
7.5.1 Preguntas a Hacerse 25
7.5.2 Red Flags Éticas 25
7.6 Código de Ética 26

8 Resumen **26**

1 Introducción

La Unidad 5 cierra el ciclo completo de análisis de datos abordando aspectos cruciales que conectan el trabajo técnico con su aplicación práctica y comunicación efectiva. Hasta ahora hemos aprendido a obtener, limpiar, explorar y modelar datos. En esta unidad nos enfocamos en:

- Llevar modelos desde el desarrollo hasta producción
- Interpretar y explicar las decisiones de modelos complejos
- Comunicar resultados de manera efectiva a diferentes audiencias
- Crear reportes reproducibles y documentación de calidad
- Considerar las implicaciones éticas del análisis de datos

Estos aspectos son frecuentemente subestimados en cursos introductorios, pero son fundamentales para el éxito profesional en ciencia de datos.

2 Ciclo de Vida de Modelos en Producción

2.1 Fases del Ciclo de Vida

El desarrollo de un modelo de machine learning no termina cuando obtenemos buenas métricas en el conjunto de validación. El ciclo completo incluye:

2.1.1 1. Desarrollo e Investigación

- **Exploración de datos:** análisis exploratorio, identificación de patrones, detección de problemas de calidad
- **Ingeniería de features:** creación, selección y transformación de variables
- **Experimentación:** probar diferentes algoritmos, hiperparámetros y estrategias
- **Validación:** evaluación rigurosa con datos no vistos, análisis de errores

Herramientas típicas: Jupyter notebooks, VS Code, bibliotecas de ML (scikit-learn, statsmodels)

2.1.2 2. Deployment (Puesta en Producción)

Una vez que tenemos un modelo que cumple los requisitos de rendimiento y negocio, debemos ponerlo a disposición de usuarios o sistemas. Esto implica:

- **Serialización:** guardar el modelo entrenado en un formato que pueda ser cargado posteriormente
- **Creación de API:** exponer el modelo mediante endpoints REST para que otros sistemas lo consuman
- **Containerización:** empaquetar modelo y dependencias (Docker) para garantizar reproducibilidad
- **Integración:** conectar el modelo con bases de datos, pipelines de datos, sistemas empresariales

Ejemplo de flujo:

1. Entrenar modelo en notebook
2. Serializar modelo con pickle/joblib
3. Crear aplicación Flask/FastAPI que carga el modelo
4. Dockerizar la aplicación
5. Deployar en servidor/cloud (AWS, Azure, GCP)

2.1.3 3. Monitoreo

Los modelos en producción deben ser monitoreados continuamente para detectar:

- **Degradación del rendimiento:** las métricas (accuracy, AUC, etc.) pueden empeorar con el tiempo
- **Data drift:** cambios en la distribución de las variables de entrada
- **Concept drift:** cambios en la relación entre predictores y variable objetivo
- **Problemas técnicos:** tiempos de respuesta, errores, disponibilidad

Métricas a monitorear:

- Performance del modelo (accuracy, precision, recall, AUC)
- Distribución de predicciones
- Distribución de features de entrada
- Latencia (tiempo de respuesta)
- Tasa de errores

2.1.4 4. Reentrenamiento y Versionado

Cuando detectamos degradación o disponemos de nuevos datos, debemos reentrenar:

- **Reentrenamiento programado:** automático cada cierto tiempo (diario, semanal, mensual)
- **Reentrenamiento disparado:** cuando las métricas caen por debajo de un umbral
- **Versionado de modelos:** mantener registro de versiones, poder hacer rollback
- **A/B testing:** comparar modelo nuevo vs antiguo en producción antes de reemplazo completo

Herramientas de versionado: MLflow, DVC, Neptune.ai, Weights & Biases

2.2 Data Drift y Concept Drift

2.2.1 Data Drift

Cambio en la distribución de las variables de entrada $P(X)$. Por ejemplo:

- Un modelo de scoring crediticio entrenado pre-pandemia puede recibir solicitudes con perfiles muy diferentes post-pandemia
- Variables demográficas cambian con el tiempo (edad, ingresos)
- Cambios estacionales en datos de ventas

Detección:

- Comparar distribuciones entre train y producción (test de Kolmogorov-Smirnov, chi-cuadrado)
- Monitorear estadísticos descriptivos (media, varianza) de cada feature
- Visualizaciones: histogramas comparativos, Q-Q plots

2.2.2 Concept Drift

Cambio en la relación entre predictores y objetivo $P(Y|X)$. El concepto que el modelo aprendió ya no es válido. Ejemplos:

- Un modelo de predicción de ventas entrenado en un mercado estable puede fallar si cambian las regulaciones
- Detección de fraude: los estafadores adaptan sus métodos
- Recomendaciones: las preferencias de usuarios cambian con el tiempo

Detección:

- Monitorear performance en producción (accuracy, precision, recall)
- Comparar predicciones vs valores reales cuando estén disponibles
- Alertas cuando métricas caen por debajo de umbrales

2.3 Consideraciones de Deployment

2.3.1 Escalabilidad

- **Latencia:** tiempo de respuesta debe ser aceptable (usualmente < 100ms para aplicaciones interactivas)
- **Throughput:** cantidad de predicciones por segundo
- **Batch vs real-time:** ¿predicciones individuales o en lote?
- **Trade-offs:** modelos más complejos pueden ser más precisos pero más lentos

2.3.2 Mantenibilidad

- **Documentación:** del modelo, features, dependencias, proceso de entrenamiento
- **Testing:** tests unitarios para preprocesamiento, validaciones de entrada/salida
- **Logging:** registrar predicciones, inputs, errores para debugging
- **Reproducibilidad:** control de versiones de código, datos, modelo

3 Serialización de Modelos

La serialización es el proceso de convertir un objeto Python (como un modelo entrenado) en un formato que puede ser almacenado en disco y posteriormente recargado.

3.1 Formatos de Serialización

3.1.1 pickle

Formato estándar de Python para serialización de objetos.

Ventajas:

- Nativo de Python, no requiere librerías adicionales
- Puede serializar casi cualquier objeto Python
- Simple de usar

Desventajas:

- Específico de Python (no portable a otros lenguajes)
- Vulnerabilidades de seguridad: no cargar pickles de fuentes no confiables
- Puede haber incompatibilidades entre versiones de librerías

Ejemplo:

```python
import pickle
from sklearn.linear_model import LogisticRegression

# Entrenar modelo
modelo = LogisticRegression()
modelo.fit(X_train, y_train)

# Guardar
with open('modelo.pkl', 'wb') as f:
    pickle.dump(modelo, f)

# Cargar
with open('modelo.pkl', 'rb') as f:
    modelo_cargado = pickle.load(f)

# Usar
predicciones = modelo_cargado.predict(X_test)
```

3.1.2 joblib

Optimización de pickle para objetos grandes con arrays NumPy (común en modelos de ML).

Ventajas:

- Más eficiente que pickle para modelos grandes
- Compresión automática
- API simple compatible con pickle

Ejemplo:

```
from joblib import dump, load

# Guardar
dump(modelo, 'modelo.joblib')

# Cargar
modelo_cargado = load('modelo.joblib')
```

3.1.3 ONNX (Open Neural Network Exchange)

Formato abierto e interoperable para modelos de ML, especialmente redes neuronales.

Ventajas:

- Portable entre frameworks (PyTorch, TensorFlow, scikit-learn)
- Portable entre lenguajes (Python, C++, Java, JavaScript)
- Optimizado para inferencia rápida

Desventajas:

- Más complejo de configurar
- No todos los modelos de scikit-learn son convertibles fácilmente

Ejemplo con scikit-learn:

```
from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType

# Definir tipo de input
initial_type = [('float_input', FloatTensorType([None, X_train.
    shape[1]]))]

# Convertir
onx = convert_sklearn(modelo, initial_types=initial_type)

# Guardar
with open("modelo.onnx", "wb") as f:
    f.write(onx.SerializeToString())
```

3.2 Buenas Prácticas de Serialización

1. **Guardar todo el pipeline:** no solo el modelo, sino también preprocesadores (scalers, encoders)
2. **Versionado:** incluir versión del modelo y de librerías en el nombre del archivo
3. **Metadata:** guardar información adicional (fecha de entrenamiento, métricas, features utilizadas)
4. **Testing post-carga:** verificar que el modelo cargado produce las mismas predicciones
5. **Compresión:** para modelos grandes, usar compresión (joblib lo hace automáticamente)

Ejemplo de serialización completa:

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from joblib import dump
import json
from datetime import datetime

# Crear pipeline completo
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier(n_estimators=100,
        random_state=42))
])

# Entrenar
pipeline.fit(X_train, y_train)

# Evaluar
accuracy = pipeline.score(X_test, y_test)

# Guardar pipeline
dump(pipeline, 'pipeline_v1.0.0.joblib')

# Guardar metadata
metadata = {
    'version': '1.0.0',
    'timestamp': datetime.now().isoformat(),
    'accuracy': accuracy,
    'features': list(X_train.columns),
    'sklearn_version': sklearn.__version__
}

with open('pipeline_v1.0.0_metadata.json', 'w') as f:
    json.dump(metadata, f, indent=2)
```

4 Interpretabilidad de Modelos

La interpretabilidad se refiere a la capacidad de entender y explicar cómo un modelo toma sus decisiones. Es crucial por:

- **Confianza:** usuarios confían más en modelos que pueden explicar

- **Debugging:** identificar errores y sesgos en el modelo
- **Cumplimiento regulatorio:** en algunos dominios (finanzas, salud) es obligatorio poder explicar decisiones
- **Mejora del modelo:** entender qué aprende el modelo ayuda a mejorar features

4.1 Modelos Interpretables vs Caja Negra

4.1.1 Modelos Inherentemente Interpretables

Regresión Lineal/Logística:

- Coeficientes indican dirección y magnitud del efecto de cada variable
- Fácil de entender: "un aumento de 1 unidad en X_1 resulta en un cambio de β_1 en Y"

Árboles de Decisión:

- Estructura visual clara: reglas if-then
- Fácil de explicar a no técnicos
- Limitación: árboles profundos pierden interpretabilidad

4.1.2 Modelos de Caja Negra

Random Forest, Gradient Boosting:

- Combinan múltiples árboles (decenas a miles)
- No hay una regla simple de interpretación
- Necesitan métodos post-hoc para interpretación

Redes Neuronales:

- Millones de parámetros
- Transformaciones no lineales complejas
- Extremadamente difíciles de interpretar directamente

SVM con kernels no lineales:

- Operan en espacios de alta dimensión
- Vectores de soporte no son directamente interpretables

4.2 Trade-off Interpretabilidad vs Performance

En general existe un trade-off:

- Modelos simples (regresión lineal, árboles pequeños) son fáciles de interpretar pero pueden tener menor performance
- Modelos complejos (random forest, redes neuronales) tienen mejor performance pero son más difíciles de interpretar

Estrategia recomendada:

1. Comenzar con modelos simples e interpretables (baseline)
2. Si el performance no es suficiente, probar modelos más complejos
3. Usar técnicas de interpretabilidad (SHAP, LIME) para explicar modelos complejos
4. Documentar claramente los trade-offs

4.3 SHAP (SHapley Additive exPlanations)

SHAP es un método unificado para explicar las salidas de modelos de machine learning, basado en valores de Shapley de la teoría de juegos cooperativos.

4.3.1 Idea Principal

Para cada predicción, SHAP calcula la contribución de cada feature al valor predicho. Formalmente:

$$f(x) = \phi_0 + \sum_{i=1}^{p} \phi_i$$

donde:

- $f(x)$ es la predicción del modelo para la instancia x
- ϕ_0 es el valor base (predicción promedio del modelo)
- ϕ_i es la contribución de la feature i (valor SHAP)

Propiedades deseables que cumple SHAP:

- **Local accuracy:** la suma de contribuciones reproduce la predicción
- **Missingness:** features ausentes no contribuyen
- **Consistency:** si un modelo cambia para depender más de una feature, su contribución aumenta

4.3.2 Interpretación de SHAP Values

- Valores positivos: la feature empuja la predicción hacia arriba
- Valores negativos: la feature empuja la predicción hacia abajo
- Magnitud: importancia de la contribución

4.3.3 Tipos de Gráficos SHAP

1. Force Plot: visualiza cómo cada feature contribuye a una predicción individual

2. Summary Plot: muestra la distribución de SHAP values para cada feature en todo el dataset

- Eje vertical: features ordenadas por importancia
- Eje horizontal: valor SHAP (impacto en predicción)
- Color: valor de la feature (alto/bajo)

3. Dependence Plot: muestra la relación entre el valor de una feature y su SHAP value

4. Waterfall Plot: descomposición aditiva de una predicción individual

4.3.4 Uso en Python

```python
import shap
from sklearn.ensemble import RandomForestClassifier

# Entrenar modelo
modelo = RandomForestClassifier()
modelo.fit(X_train, y_train)

# Crear explainer
explainer = shap.TreeExplainer(modelo)

# Calcular SHAP values
shap_values = explainer.shap_values(X_test)

# Visualizar
# Summary plot (global)
shap.summary_plot(shap_values[1], X_test)

# Force plot (individual, primera instancia)
shap.force_plot(explainer.expected_value[1],
                shap_values[1][0],
                X_test.iloc[0])

# Dependence plot
shap.dependence_plot("feature_name", shap_values[1], X_test)
```

4.4 LIME (Local Interpretable Model-agnostic Explanations)

LIME explica predicciones individuales aproximando localmente el modelo con un modelo interpretable simple.

4.4.1 Idea Principal

Para explicar una predicción:

1. Generar perturbaciones (variaciones) del punto a explicar
2. Obtener predicciones del modelo original para estas perturbaciones
3. Entrenar un modelo interpretable simple (e.g., regresión lineal) en las perturbaciones, ponderando más las cercanas al punto original
4. El modelo simple aproxima localmente el comportamiento del modelo complejo

Analogía: es como aproximar una función compleja no lineal con una recta tangente en un punto específico.

4.4.2 Ventajas de LIME

- **Model-agnostic:** funciona con cualquier modelo de caja negra
- **Explicaciones locales:** se enfoca en una predicción específica
- **Interpretable:** usa modelos lineales como explicadores

4.4.3 Desventajas de LIME

- **Inestabilidad:** pequeños cambios en el punto pueden cambiar mucho la explicación
- **Dependencia de hiperparámetros:** tamaño de vecindad, kernel
- **No garantiza propiedades globales:** solo válido localmente

4.4.4 Uso en Python

```python
import lime
import lime.lime_tabular

# Crear explainer
explainer = lime.lime_tabular.LimeTabularExplainer(
    X_train.values,
    feature_names=X_train.columns,
    class_names=['Clase 0', 'Clase 1'],
    mode='classification'
)

# Explicar una instancia
i = 0  #  ndice  de la instancia
exp = explainer.explain_instance(
    X_test.values[i],
    modelo.predict_proba,
    num_features=10
)

# Visualizar
exp.show_in_notebook()

# O como gr fico
exp.as_pyplot_figure()
```

4.5 Partial Dependence Plots (PDP)

Los PDP muestran el efecto marginal de una feature sobre la predicción, manteniendo las demás constantes.

4.5.1 Cálculo

Para una feature X_j:

$$PD_{X_j}(x_j) = \mathbb{E}_{X_{-j}}[f(x_j, X_{-j})] = \frac{1}{n}\sum_{i=1}^{n} f(x_j, x_{-j}^{(i)})$$

Es decir, fijamos X_j en un valor x_j, promediamos las predicciones sobre todos los valores observados de las demás features.

4.5.2 Interpretación

- Eje horizontal: valores de la feature
- Eje vertical: predicción promedio
- La curva muestra cómo cambia la predicción al variar la feature

- Pendiente positiva: relación positiva
- Pendiente negativa: relación negativa
- Curva plana: la feature no afecta la predicción

4.5.3 Limitaciones

- **Supuesto de independencia:** asume que las features son independientes (raramente cierto)
- **No captura interacciones:** promedia sobre todas las combinaciones
- **Puede ser engañoso:** si hay interacciones fuertes

Solución para interacciones: Individual Conditional Expectation (ICE) plots, que muestran el efecto para cada instancia individualmente.

4.5.4 Uso en Python

```
from sklearn.inspection import PartialDependenceDisplay

# PDP para una feature
fig, ax = plt.subplots(figsize=(10, 6))
PartialDependenceDisplay.from_estimator(
    modelo, X_train,
    features=['feature_name'],
    ax=ax
)
plt.show()

# PDP para m ltiples features
fig, ax = plt.subplots(figsize=(12, 4))
PartialDependenceDisplay.from_estimator(
    modelo, X_train,
    features=['feature1', 'feature2', ('feature1', 'feature2')
        ],
    ax=ax
)
plt.show()
```

4.6 Feature Importance

Mide la importancia relativa de cada feature para el modelo.

4.6.1 Importancia en Árboles y Random Forest

Para modelos basados en árboles, la importancia se calcula como la disminución promedio en impureza (Gini o entropía) causada por cada feature.

Ventajas:

- Rápida de calcular
- Incorporada en scikit-learn

Desventajas:

- Sesgada hacia features con alta cardinalidad
- No distingue correlación de causalidad

```
# Obtener importancias
importances = modelo.feature_importances_

# Visualizar
indices = np.argsort(importances)[::-1]
plt.figure(figsize=(10, 6))
plt.bar(range(X_train.shape[1]), importances[indices])
plt.xticks(range(X_train.shape[1]),
           X_train.columns[indices],
           rotation=90)
plt.title("Feature Importances")
plt.tight_layout()
plt.show()
```

4.6.2 Permutation Importance

Mide el aumento en el error del modelo cuando se permutan aleatoriamente los valores de una feature (rompiendo su relación con el objetivo).

Ventajas:

- Model-agnostic
- No sesgada por cardinalidad
- Refleja importancia real para predicción

Desventajas:

- Más lenta de calcular
- Puede ser confusa con features correlacionadas

```
from sklearn.inspection import permutation_importance

# Calcular
perm_importance = permutation_importance(
    modelo, X_test, y_test,
    n_repeats=10, random_state=42
)

# Visualizar
sorted_idx = perm_importance.importances_mean.argsort()[::-1]
plt.figure(figsize=(10, 6))
plt.boxplot(perm_importance.importances[sorted_idx].T,
            labels=X_test.columns[sorted_idx],
            vert=False)
plt.xlabel("Decrease in accuracy")
plt.tight_layout()
plt.show()
```

5 Comunicación de Resultados

La comunicación efectiva es tan importante como el análisis técnico. Los mejores modelos son inútiles si no podemos comunicar sus hallazgos de manera clara y convincente.

5.1 Principios de Comunicación Efectiva

5.1.1 1. Conocer a la Audiencia

Audiencia técnica (científicos de datos, ingenieros):

- Puede incluir detalles técnicos (hiperparámetros, métricas específicas)
- Aprecian gráficos de diagnóstico (residuos, curvas ROC)
- Interesados en reproducibilidad y metodología

Audiencia de negocio (gerentes, ejecutivos):

- Enfoque en impacto, ROI, decisiones accionables
- Visualizaciones simples y claras
- Lenguaje no técnico, evitar jerga
- Destacar resultados clave y recomendaciones

Audiencia mixta:

- Resumen ejecutivo al inicio
- Detalles técnicos en apéndices
- Gráficos intuitivos con explicaciones breves

5.1.2 2. Estructura Clara

Reporte típico:

1. **Resumen ejecutivo (1 página):** problema, metodología, resultados clave, recomendaciones
2. **Introducción:** contexto, objetivos, preguntas de investigación
3. **Datos:** fuentes, tamaño, calidad, limitaciones
4. **Metodología:** análisis exploratorio, features, modelos probados, validación
5. **Resultados:** métricas, comparaciones, hallazgos principales
6. **Conclusiones:** interpretación, implicaciones, recomendaciones
7. **Limitaciones:** supuestos, sesgos, incertidumbre
8. **Próximos pasos:** mejoras futuras, áreas de investigación
9. **Apéndices:** detalles técnicos, código, tablas completas

5.1.3 3. Visualizaciones Efectivas

Principios:

- **Simplicidad:** un gráfico, un mensaje
- **Claridad:** ejes etiquetados, leyendas, títulos descriptivos
- **Honestidad:** no distorsionar (escalas apropiadas, barras desde cero)
- **Accesibilidad:** colores apropiados para daltónicos, tamaño de fuente legible
- **Consistencia:** mismo estilo a lo largo del reporte

Elegir el gráfico apropiado:

- Comparación de categorías: barras, columnas
- Evolución temporal: líneas
- Distribución: histogramas, boxplots, violines
- Relación entre dos variables: scatter plots
- Proporciones: pie charts (usar con moderación), treemaps
- Múltiples variables: heatmaps, parallel coordinates

5.1.4 4. Narrar una Historia

Los datos por sí solos no son convincentes. Necesitamos construir una narrativa:

- **Contexto:** ¿por qué es importante este análisis?
- **Problema:** ¿qué pregunta estamos respondiendo?
- **Viaje:** ¿qué descubrimos en el camino?
- **Clímax:** hallazgo principal
- **Resolución:** ¿qué debemos hacer con esta información?

Técnicas narrativas:

- Comenzar con un ejemplo concreto o anécdota
- Usar analogías para conceptos complejos
- Anticipar y responder objeciones
- Terminar con un llamado a la acción claro

5.2 Reportar Incertidumbre

Todo modelo tiene incertidumbre. Es crucial comunicarla honestamente:

- **Intervalos de confianza:** no solo puntos estimados
- **Errores de predicción:** RMSE, MAE contextualizados
- **Limitaciones de los datos:** sesgos de muestreo, datos faltantes
- **Supuestos del modelo:** qué asumimos y qué tan razonables son

Ejemplo de buena práctica:
"El modelo predice una tasa de churn del 15% con un intervalo de confianza del 95% de [12%, 18%]. Esta estimación asume que el comportamiento futuro de los clientes será similar al histórico. Cambios en el mercado o en nuestros productos podrían afectar esta predicción."

5.3 Lenguaje Claro y Preciso

Evitar:

- Jerga innecesaria ("hiperplano de máximo margen")
- Siglas no definidas (AUC, RMSE sin explicar)
- Lenguaje vago ("el modelo funciona bien")
- Causalidad cuando solo hay correlación ("X causa Y")

Preferir:

- Términos simples y descriptivos
- Definiciones breves en primera mención
- Cuantificación precisa ("el modelo clasifica correctamente el 85% de los casos")
- Lenguaje asociativo ("X está asociado con Y", "X predice Y")

6 Reportes Reproducibles

La reproducibilidad es fundamental en ciencia de datos. Un análisis reproducible puede ser ejecutado por otra persona y obtener los mismos resultados.

6.1 Jupyter Notebooks

Jupyter combina código, visualizaciones y narrativa en un solo documento interactivo.

6.1.1 Ventajas

- **Interactividad:** ejecutar código celda por celda
- **Integración:** código, texto (Markdown), ecuaciones (LaTeX), imágenes
- **Compartible:** formato JSON, puede versionarse con git
- **Exportable:** HTML, PDF, presentaciones
- **Popular:** ampliamente usado en industria y academia

6.1.2 Buenas Prácticas

1. **Orden lógico:** celdas ejecutables de arriba a abajo secuencialmente
2. **Restart & Run All:** asegurarse que el notebook funcione desde cero
3. **Markdown generoso:** explicar qué hace cada sección
4. **Visualizaciones inline:** configurar `%matplotlib inline`
5. **Control de versiones:** hacer commits frecuentes
6. **Evitar outputs grandes:** limpiar antes de commitear
7. **Gestión de dependencias:** incluir requirements.txt o environment.yml

6.1.3 Estructura Recomendada

1. **Título y resumen:** qué hace este notebook
2. **Imports:** todas las librerías al inicio
3. **Configuración:** seeds, parámetros, paths
4. **Carga de datos:** con verificaciones básicas
5. **Exploración:** análisis exploratorio
6. **Preprocesamiento:** limpieza, transformaciones
7. **Modelado:** entrenamiento, validación
8. **Evaluación:** métricas, gráficos
9. **Conclusiones:** hallazgos principales
10. **Próximos pasos:** mejoras futuras

6.1.4 Exportación

HTML (recomendado para compartir):

```
jupyter nbconvert --to html notebook.ipynb
```

PDF (requiere LaTeX instalado):

```
jupyter nbconvert --to pdf notebook.ipynb
```

Presentación (slides con reveal.js):

```
jupyter nbconvert --to slides notebook.ipynb --post serve
```

6.2 R Markdown

R Markdown permite combinar código R, narrativa y análisis en documentos reproducibles.

6.2.1 Estructura

Un archivo .Rmd tiene tres componentes:

1. **Encabezado YAML:** metadatos y opciones

```
---
title: "An lisis de Ventas Q1 2026"
author: "Jos  Bavio"
date: "2026-03-05"
output:
  html_document:
    toc: true
    toc_float: true
    theme: cosmo
---
```

2. **Texto Markdown:** narrativa, explicaciones
3. **Chunks de código R:** entre ```{r} y ```

6.2.2 Opciones de Chunks

Controlan cómo se ejecuta y muestra el código:

- `echo=FALSE`: no mostrar código, solo output
- `eval=FALSE`: mostrar código pero no ejecutar
- `include=FALSE`: ejecutar pero no mostrar nada
- `message=FALSE, warning=FALSE`: suprimir mensajes
- `fig.width=8, fig.height=6`: tamaño de gráficos
- `cache=TRUE`: cachear resultados (útil para cálculos lentos)

6.2.3 Formatos de Salida

HTML (interactivo, con gráficos dinámicos):

```
output: html_document
```

PDF (vía LaTeX):

```
output: pdf_document
```

Word:

```
output: word_document
```

Presentaciones:

```
output:
  ioslides_presentation  # Google IO style
  slidy_presentation     # W3C style
  beamer_presentation    # LaTeX Beamer
```

6.2.4 Parámetros

R Markdown permite crear reportes parametrizados:

```
---
title: "Reporte de Ventas"
params:
  region: "Norte"
  a o: 2026
---

## Ventas en `r params$region` - `r params$a o`

```{r}
datos_filtrados <- datos %>%
  filter(region == params$region, a o == params$a o)
```
```

Renderizar con parámetros diferentes:

```
rmarkdown::render("reporte.Rmd",
                  params = list(region = "Sur", a o = 2025))
```

Esto permite automatizar reportes para diferentes regiones, períodos, etc.

6.3 Control de Versiones

Git es esencial para reproducibilidad:

- **Historial completo:** registra todos los cambios
- **Colaboración:** múltiples personas trabajando simultáneamente
- **Branching:** experimentar sin afectar código principal
- **Rollback:** volver a versiones anteriores

Workflow básico:

```
# Inicializar repositorio
git init

# Agregar archivos
git add notebook.ipynb

# Commit
git commit -m "Agregar an lisis exploratorio"

# Subir a remoto (GitHub, GitLab)
git push origin main
```

.gitignore para data science:

```
# Datos
data/
*.csv
*.xlsx

# Modelos entrenados
*.pkl
*.joblib
```

```
*.h5

# Notebooks con outputs
.ipynb_checkpoints/

# Entornos virtuales
venv/
env/

# IDE
.vscode/
.idea/
```

6.4 Gestión de Dependencias

Documentar qué librerías y versiones se usaron:

Python - requirements.txt:

```
pandas==2.0.1
numpy==1.24.3
scikit-learn==1.2.2
matplotlib==3.7.1
seaborn==0.12.2
```

Generar automáticamente:

```
pip freeze > requirements.txt
```

Instalar desde requirements:

```
pip install -r requirements.txt
```

Python - conda environment.yml:

```
name: mad1
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.10
  - pandas=2.0.1
  - scikit-learn=1.2.2
  - matplotlib=3.7.1
```

R - renv:

```
# Inicializar
renv::init()

# Snapshot (guardar estado actual)
renv::snapshot()

# Restore (reinstalar desde snapshot)
renv::restore()
```

7 Ética en Análisis de Datos

El análisis de datos tiene poder para afectar vidas. Con este poder viene responsabilidad.

7.1 Sesgo en Datos y Modelos

7.1.1 Fuentes de Sesgo

1. Sesgo de Selección:

- Datos no representativos de la población objetivo
- Ejemplo: entrenar modelo de contratación con datos históricos sesgados

2. Sesgo de Medición:

- Errores sistemáticos en cómo se recolectan los datos
- Ejemplo: encuestas telefónicas excluyen a quienes no tienen teléfono

3. Sesgo Algorítmico:

- El algoritmo amplifica sesgos existentes
- Ejemplo: sistema de recomendación que refuerza burbujas de filtro

4. Sesgo de Confirmación:

- Analistas buscan/interpretan datos de manera que confirme sus creencias
- Ejemplo: cherry-picking resultados favorables

7.1.2 Consecuencias del Sesgo

- **Discriminación:** decisiones injustas contra grupos protegidos
- **Profecía autocumplida:** modelo sesgado perpetúa desigualdad
- **Pérdida de confianza:** en sistemas algorítmicos
- **Daño legal:** violación de leyes anti-discriminación

Caso de estudio: COMPAS (Correctional Offender Management Profiling for Alternative Sanctions)

Un sistema de evaluación de riesgo de reincidencia usado en el sistema judicial de EEUU fue encontrado con sesgo racial: predecía falsamente alta reincidencia para acusados afroamericanos.

7.1.3 Mitigación de Sesgo

1. **Auditoría de datos:**
   - ¿Los datos son representativos?
   - ¿Hay subrepresentación de algún grupo?
   - ¿Cómo se recolectaron los datos?
2. **Análisis de equidad (fairness):**
   - Métricas de equidad: disparate impact, equalized odds
   - Evaluar performance por subgrupo (género, raza, edad)
   - Detectar diferencias sistemáticas
3. **Técnicas de mitigación:**

- Pre-procesamiento: rebalancear datos, eliminar variables sensibles
- In-processing: regularización de fairness, adversarial debiasing
- Post-procesamiento: ajustar umbrales por grupo

4. **Diversidad en equipos:**
   - Perspectivas diversas ayudan a identificar sesgos
   - Incluir stakeholders afectados en el proceso
5. **Transparencia y documentación:**
   - Documentar limitaciones conocidas
   - Datasheets for datasets, Model Cards

7.2 Fairness y Discriminación Algorítmica

Existen múltiples definiciones de fairness, a menudo incompatibles entre sí:

7.2.1 1. Demographic Parity (Paridad Demográfica)

El modelo predice la misma proporción de resultados positivos para todos los grupos:

$$P(\hat{Y} = 1|A = a) = P(\hat{Y} = 1|A = b)$$

donde A es el atributo sensible (raza, género).
Ejemplo: mismo porcentaje de hombres y mujeres aprobados para un crédito.

7.2.2 2. Equalized Odds

Las tasas de verdaderos positivos y falsos positivos son iguales entre grupos:

$$P(\hat{Y} = 1|Y = 1, A = a) = P(\hat{Y} = 1|Y = 1, A = b)$$

$$P(\hat{Y} = 1|Y = 0, A = a) = P(\hat{Y} = 1|Y = 0, A = b)$$

Ejemplo: el modelo identifica correctamente a candidatos cualificados y no cualificados por igual en ambos grupos.

7.2.3 3. Predictive Parity

El valor predictivo positivo es igual entre grupos:

$$P(Y = 1|\hat{Y} = 1, A = a) = P(Y = 1|\hat{Y} = 1, A = b)$$

Ejemplo: entre quienes el modelo predice reincidencia, el porcentaje que realmente reincide es el mismo en ambos grupos.

7.2.4 Tensiones entre Definiciones

En general, no es posible satisfacer múltiples definiciones simultáneamente. Debemos elegir basándonos en el contexto:

- **Contratación:** demographic parity puede ser apropiada (cuotas)
- **Justicia penal:** equalized odds (evitar discriminación en precisión)
- **Medicina:** predictive parity (confianza en diagnósticos)

7.3 Privacidad y Protección de Datos

7.3.1 Principios

1. **Minimización de datos:** recolectar solo lo necesario
2. **Propósito específico:** usar datos solo para lo autorizado
3. **Consentimiento informado:** las personas deben saber qué se recolecta y cómo se usa
4. **Transparencia:** derecho a saber qué datos se tienen
5. **Derecho al olvido:** poder eliminar datos personales
6. **Seguridad:** proteger datos contra acceso no autorizado

7.3.2 Regulaciones

GDPR (General Data Protection Regulation, Unión Europea):

- Protección de datos personales de ciudadanos UE
- Consentimiento explícito requerido
- Derecho a explicación de decisiones algorítmicas
- Multas severas por incumplimiento

CCPA (California Consumer Privacy Act):

- Derechos de consumidores sobre sus datos
- Opt-out de venta de datos
- Transparencia en recolección y uso

Argentina - Ley 25.326 de Protección de Datos Personales:

- Regula tratamiento de datos personales
- Consentimiento del titular
- Derecho de acceso, rectificación y supresión

7.3.3 Técnicas de Privacidad

1. Anonimización:

- Eliminar identificadores directos (nombre, DNI, dirección)
- Problema: reidentificación mediante quasi-identificadores

2. K-anonimity:

- Cada registro es indistinguible de al menos $k - 1$ otros
- Generalización y supresión de atributos

3. Differential Privacy:

- Agregar ruido aleatorio a los resultados

- Garantía matemática de privacidad individual
- Trade-off entre privacidad y utilidad

4. Federated Learning:

- Entrenar modelos sin centralizar datos
- Datos permanecen en dispositivos locales
- Solo se comparten actualizaciones del modelo

7.4 Responsabilidad en Uso de Modelos Predictivos

7.4.1 Limitaciones y Comunicación

- **Documentar supuestos:** qué condiciones deben cumplirse para que el modelo funcione
- **Reportar incertidumbre:** intervalos de confianza, márgenes de error
- **Advertir sobre mal uso:** para qué NO debe usarse el modelo
- **Actualización periódica:** modelos se vuelven obsoletos

7.4.2 Decisiones Humanas vs Algorítmicas

Cuándo usar algoritmos:

- Decisiones repetitivas, alto volumen
- Criterios objetivos claramente definidos
- Datos de alta calidad disponibles
- Consecuencias moderadas de errores

Cuándo mantener humanos en el loop:

- Decisiones con alto impacto individual (salud, justicia)
- Contextos complejos que requieren juicio
- Situaciones novedosas no vistas en entrenamiento
- Casos donde explicación y empatía son importantes

Enfoque recomendado: "Human-in-the-loop"

- Modelo hace recomendación
- Humano toma decisión final
- Humano puede anular al modelo con justificación
- Feedback de decisiones humanas mejora modelo

7.4.3 Auditoría y Rendición de Cuentas

- **Auditorías regulares:** revisar performance, sesgo, errores
- **Trazabilidad:** registrar decisiones y sus justificaciones
- **Responsables identificados:** quién responde por el sistema
- **Mecanismos de apelación:** las personas deben poder disputar decisiones
- **Evaluaciones de impacto:** antes del despliegue, evaluar riesgos

7.5 Consideraciones Éticas en Proyectos

7.5.1 Preguntas a Hacerse

Antes de empezar un proyecto de análisis de datos:

1. **¿Tenemos derecho a estos datos?** ¿Hubo consentimiento informado?
2. **¿Para qué se usarán los resultados?** ¿Podría usarse para propósitos dañinos?
3. **¿Quién se beneficia y quién podría ser perjudicado?**
4. **¿Los datos son representativos?** ¿Hay sesgos conocidos?
5. **¿Qué tan confiables son las predicciones?** ¿Cuál es el margen de error?
6. **¿Hay alternativas no algorítmicas?** ¿Son preferibles?
7. **¿Quién será responsable si algo sale mal?**
8. **¿Podemos explicar las decisiones?** ¿A quién, cómo?

7.5.2 Red Flags Éticas

Señales de alerta que indican problemas éticos potenciales:

- Cliente solicita "eliminar variables sensibles pero mantener performance"
- Objetivos vagos o cambiantes ("solo queremos ver qué encontramos")
- Presión para reportar solo resultados favorables
- Falta de transparencia sobre uso final
- Datos obtenidos de manera cuestionable
- Stakeholders afectados no consultados
- Consecuencias de error asimétricas (falsos negativos mucho peores que falsos positivos, o viceversa)

7.6 Código de Ética

Organizaciones profesionales han desarrollado códigos de ética:

ACM Code of Ethics (Association for Computing Machinery):

- Contribuir al bienestar humano y social
- Evitar daño
- Ser honesto y confiable
- Ser justo y no discriminar
- Respetar privacidad
- Honrar confidencialidad

Principios a seguir como científicos de datos:

1. **Integridad:** reportar hallazgos honestamente, incluso si no son favorables
2. **Transparencia:** documentar limitaciones, supuestos, incertidumbres
3. **Responsabilidad:** considerar impacto potencial de nuestro trabajo
4. **Equidad:** esforzarse por crear sistemas justos y no discriminatorios
5. **Privacidad:** proteger datos personales y respetar consentimiento
6. **Competencia:** reconocer límites de nuestra expertise, buscar ayuda cuando sea necesario
7. **Beneficencia:** buscar que nuestro trabajo beneficie a la sociedad

8 Resumen

La Unidad 5 completa el ciclo de análisis de datos enfocándose en aspectos cruciales pero frecuentemente descuidados:

- **Ciclo de vida en producción:** llevar modelos desde desarrollo hasta deployment, monitoreo y reentrenamiento
- **Serialización:** guardar y cargar modelos con pickle, joblib, ONNX
- **Interpretabilidad:** explicar decisiones de modelos complejos con SHAP, LIME, PDP
- **Comunicación:** transmitir hallazgos de manera efectiva a audiencias técnicas y no técnicas
- **Reportes reproducibles:** Jupyter notebooks y R Markdown para documentación ejecutable
- **Ética:** reconocer y mitigar sesgos, proteger privacidad, ser responsables en el uso de datos

Estos aspectos son fundamentales para el éxito profesional. Un modelo técnicamente perfecto es inútil si no puede desplegarse, explicarse o comunicarse adecuadamente. Y un análisis brillante puede causar daño si no se consideran sus implicaciones éticas.

Como científicos de datos, tenemos la responsabilidad de:

- Crear modelos que funcionen en el mundo real, no solo en notebooks
- Explicar nuestros hallazgos de manera clara y honesta

- Documentar nuestro trabajo para que otros puedan reproducirlo y auditarlo
- Considerar el impacto de nuestras decisiones técnicas en las personas
- Abogar por el uso ético de datos y algoritmos

La ciencia de datos es poder. Usemos ese poder sabiamente.